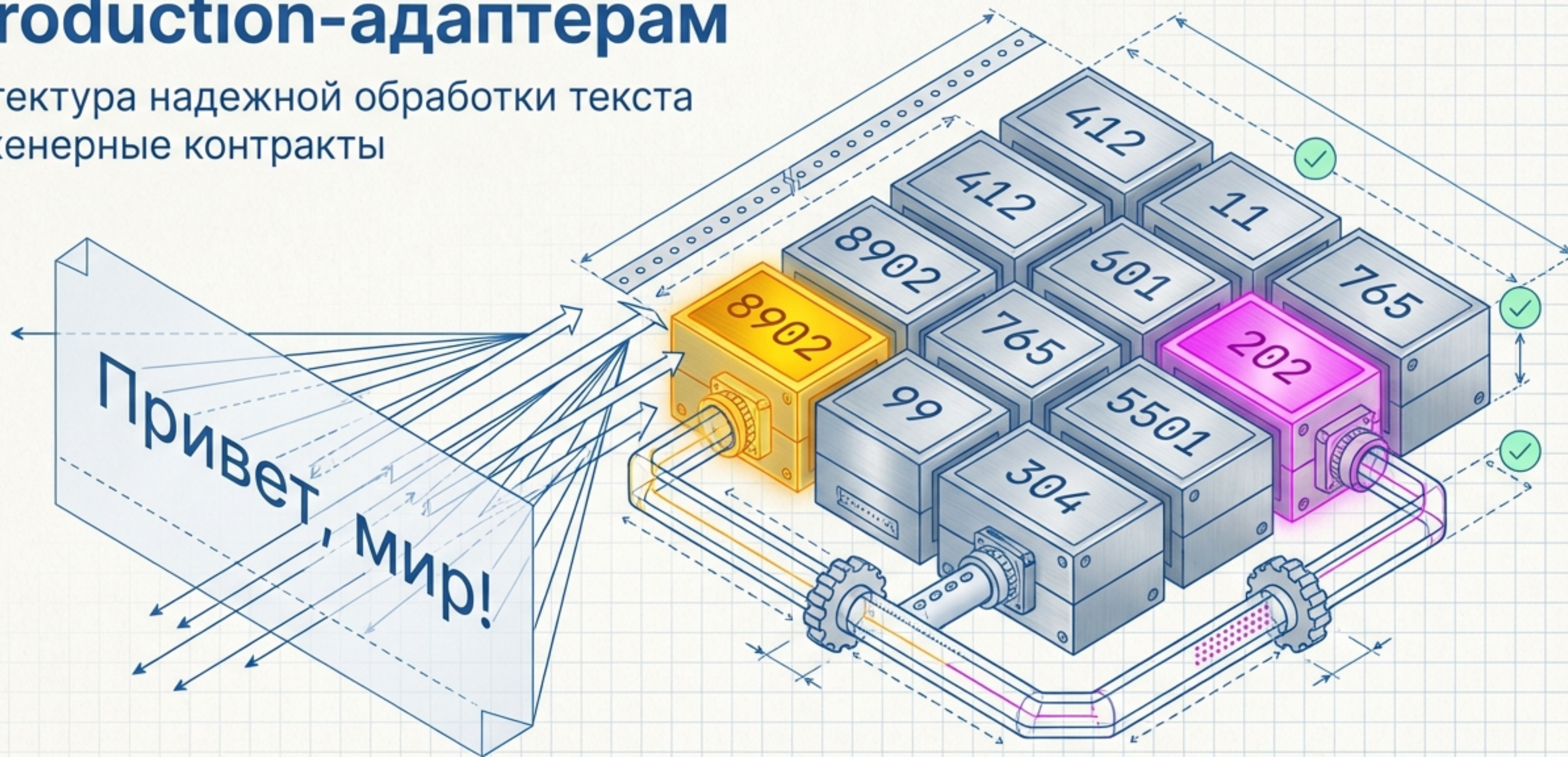
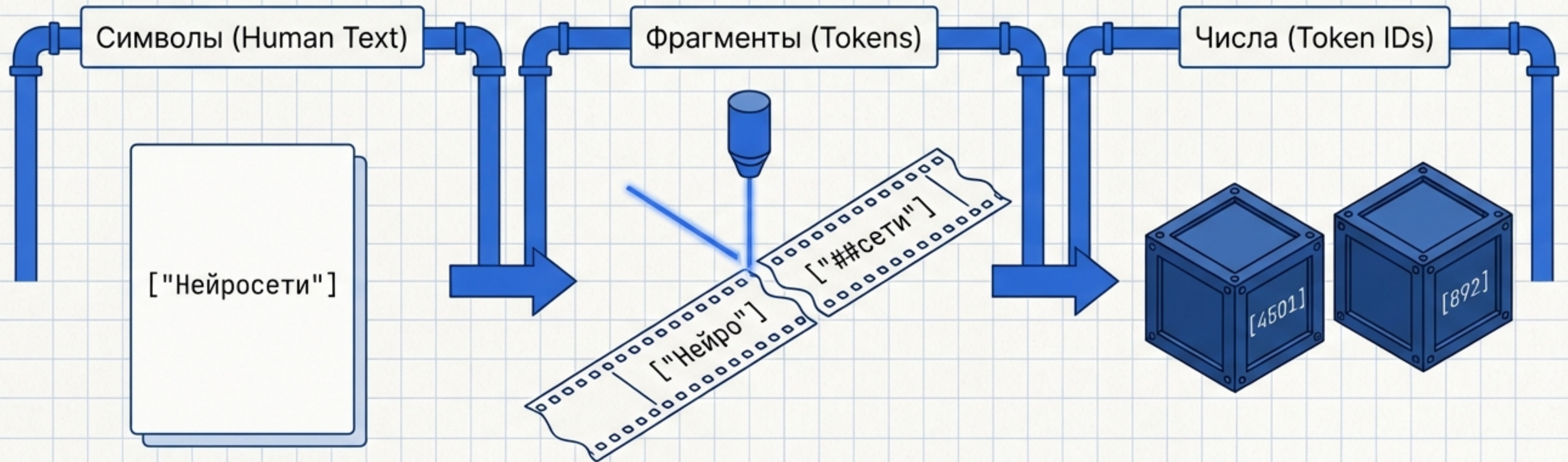


# Токенизация в LLM: От теории к production-адаптерам

Архитектура надежной обработки текста и инженерные контракты



# Мост между человеком и машиной



**Ключевой инсайт:** Модель не читает слова. Она выполняет математические операции над индексами из своего фиксированного словаря. Токенизация — это полная процедура перехода от текста к token ids.

# Три уровня абстракции токенизации

## 1. Лингвистический

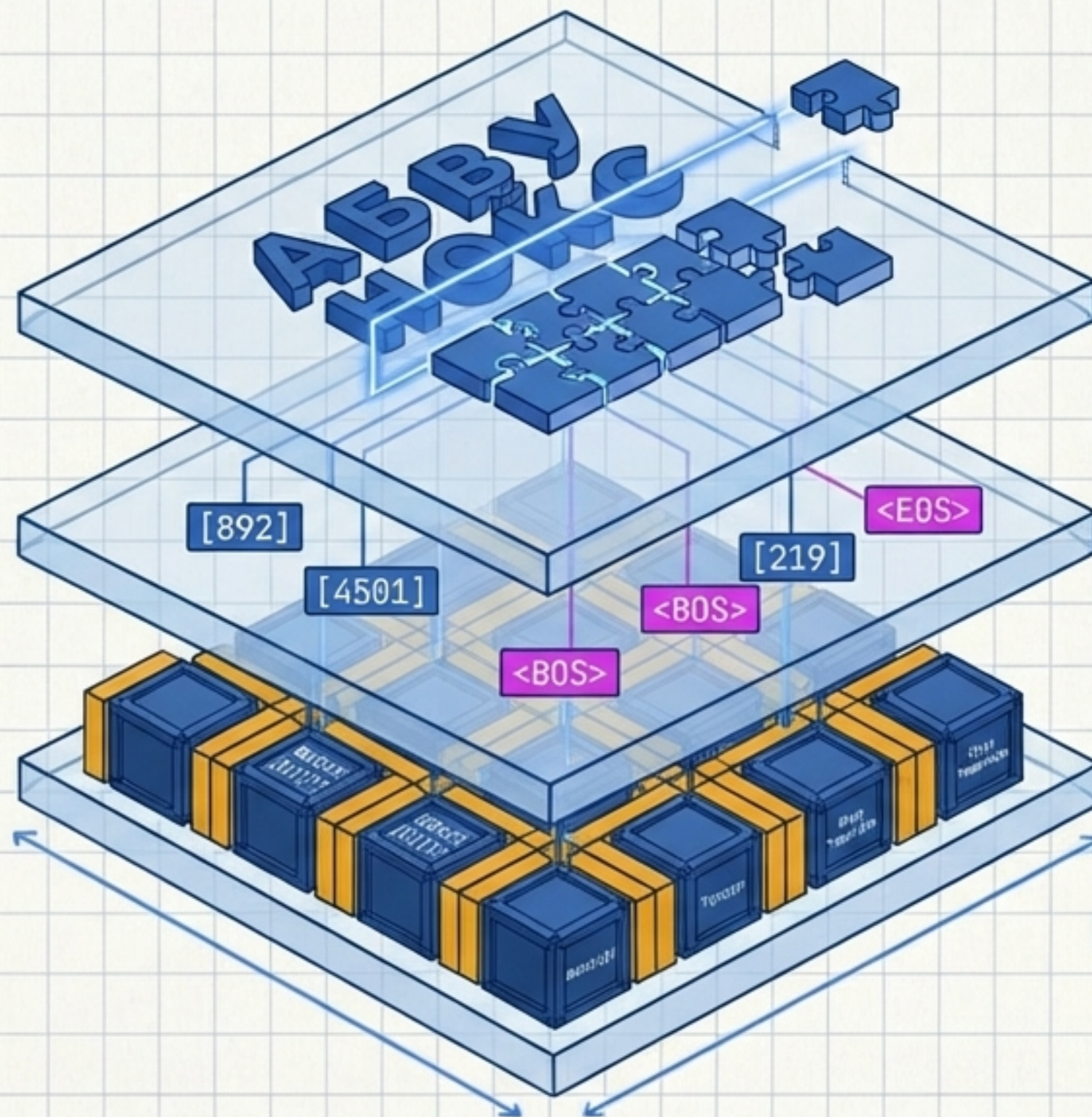
Разбиение текста на минимальные осмысленные фрагменты (subwords).

## 2. Алгоритмический

Сопоставление фрагментов с числами, внедрение специальных токенов (BOS, EOS).

## 3. Инженерный

Выравнивание батчей (padding), строгая типизация, шаблоны чата и контракты.



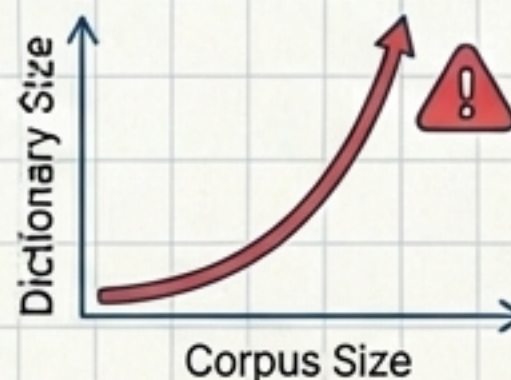
Хорошая инфраструктура учитывает все три уровня.

# Почему наивный 'split' не работает

Размер словаря

## Разделение по пробелам

Экспоненциальный взрыв из-за различных словоформ и окончаний.



Неизвестные слова (OOV)

Ошибки на URL, артикулах, опечатках.

[http://!typo]

[http://error\_404.com/item#123B-typo]

Многоязычность

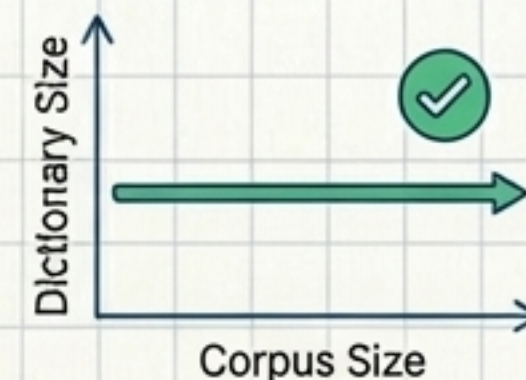
Ломается на языках без пробелов (китайский, японский).

中国語語彙的題  
文語彙的題  
愛護你國語  
文語彙的題

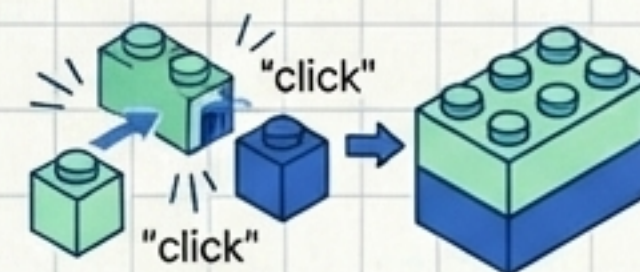
CJK Text Input Error

## Subword-токенизация

Компактный и предсказуемый.



Редкие слова собираются из известных частей.



[sub] [word] [assembly]

Универсальная обработка смешанных текстов.



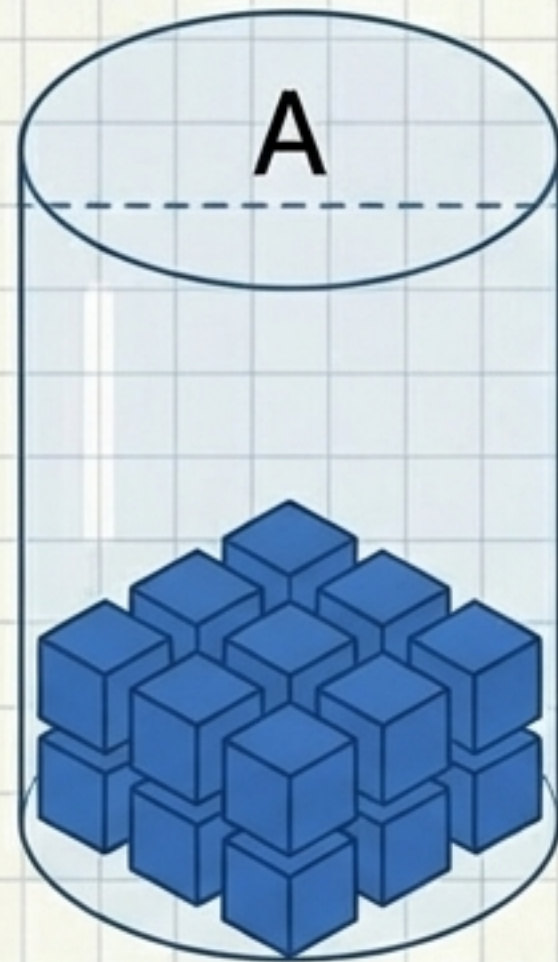
[Mixed] [文] [Текст]

# Анатомия токена: Токен ≠ Слово

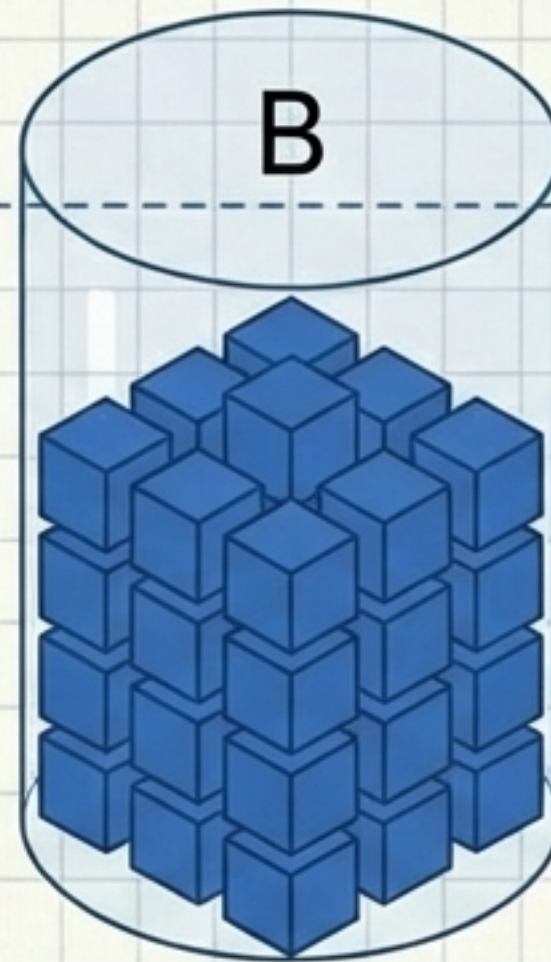


**Критическое правило:** Разложение всегда зависит от конкретной модели. Замена токенизатора без переобучения модели гарантированно ломает систему.

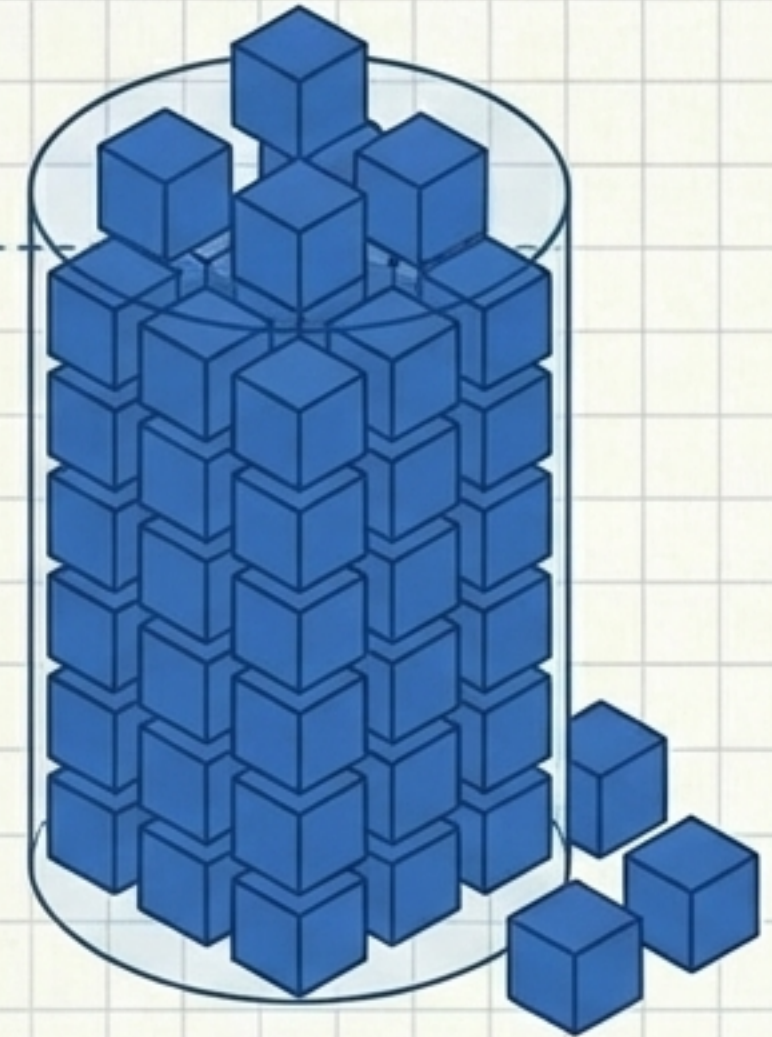
# Парадокс длины: Символы против Токенов



Английский текст  
1 токен  $\approx$  4 символа



Код / JSON / URL  
Средний расход



Русский тех. текст  
Высокий расход токенов

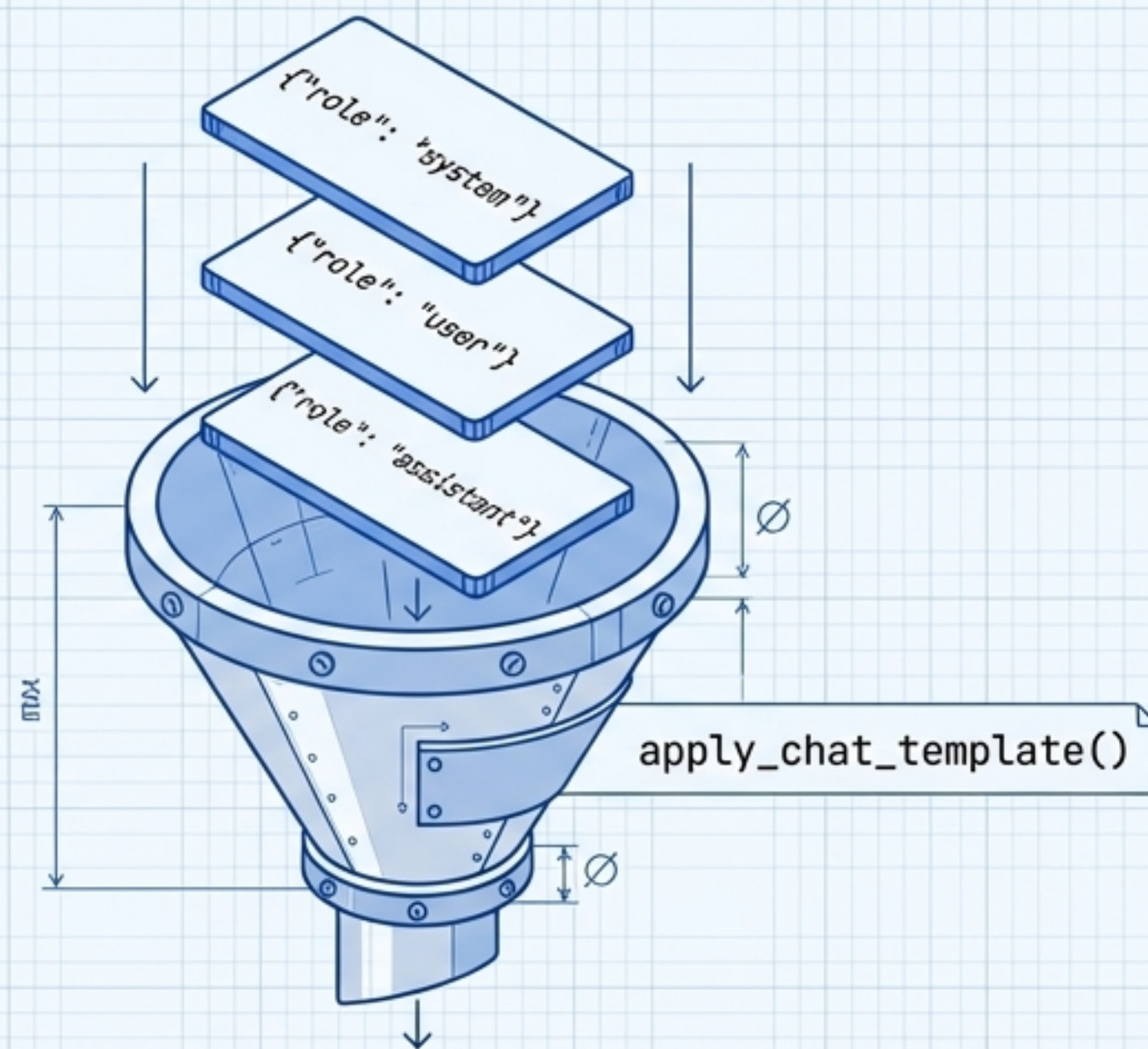
Ограничения модели и стоимость инференса задаются исключительно в токенах.  
Длина последовательности — это главный физический ресурс LLM.

# Служебные токены: Каркас последовательности



Специальные токены обозначают не содержимое текста, а его физическую структуру и границы. Без них модель не понимает, где заканчивается мысль.

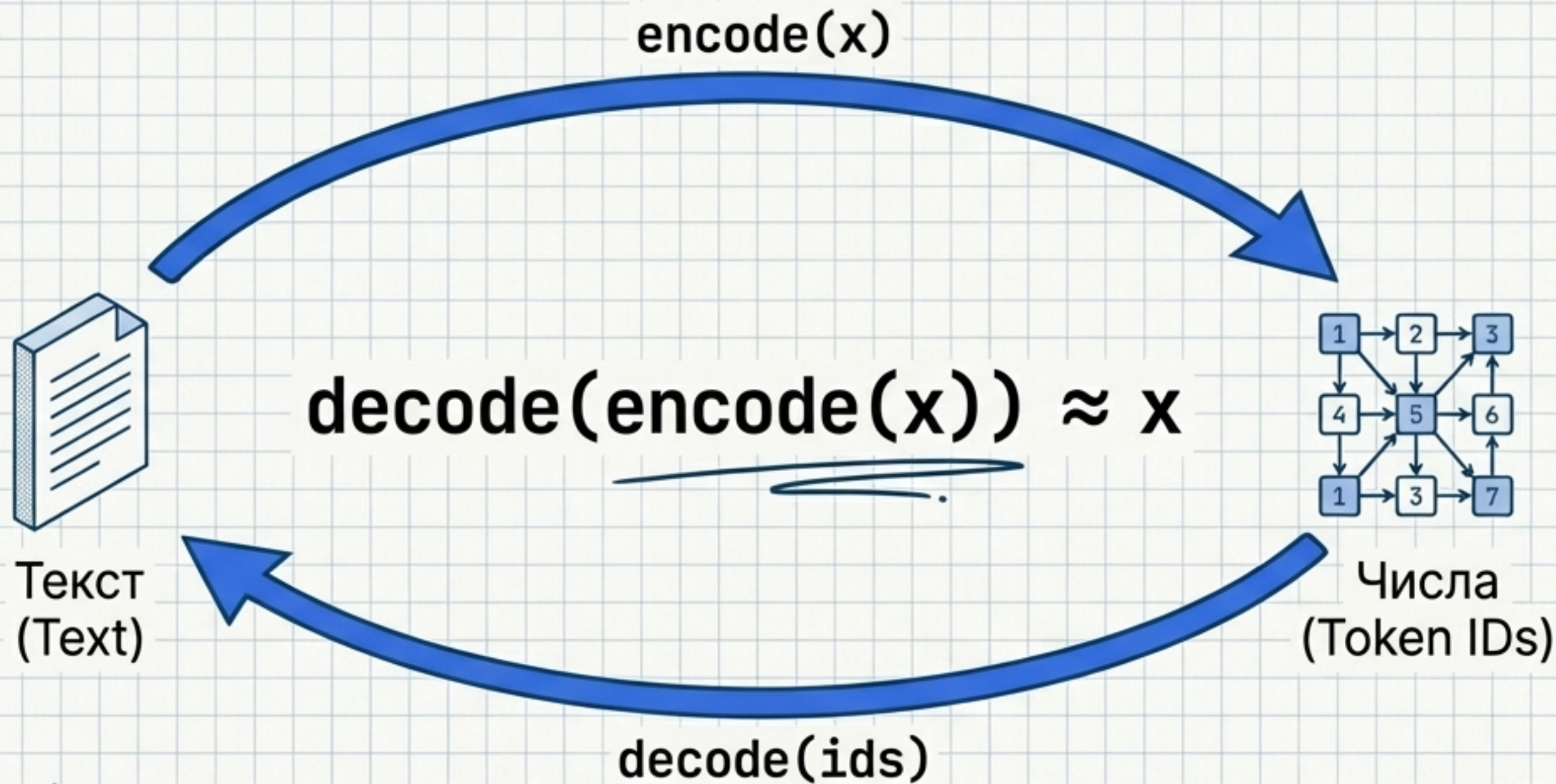
# Линеаризация диалога (Chat Templates)



`<|im_start|>` user `<|im_end|>` `<|im_start|>` assistant

Токенизация чата — это семантическая сериализация структуры. Использование неверного шаблона ломает логику инструкционного следования.

# Принцип практической обратимости



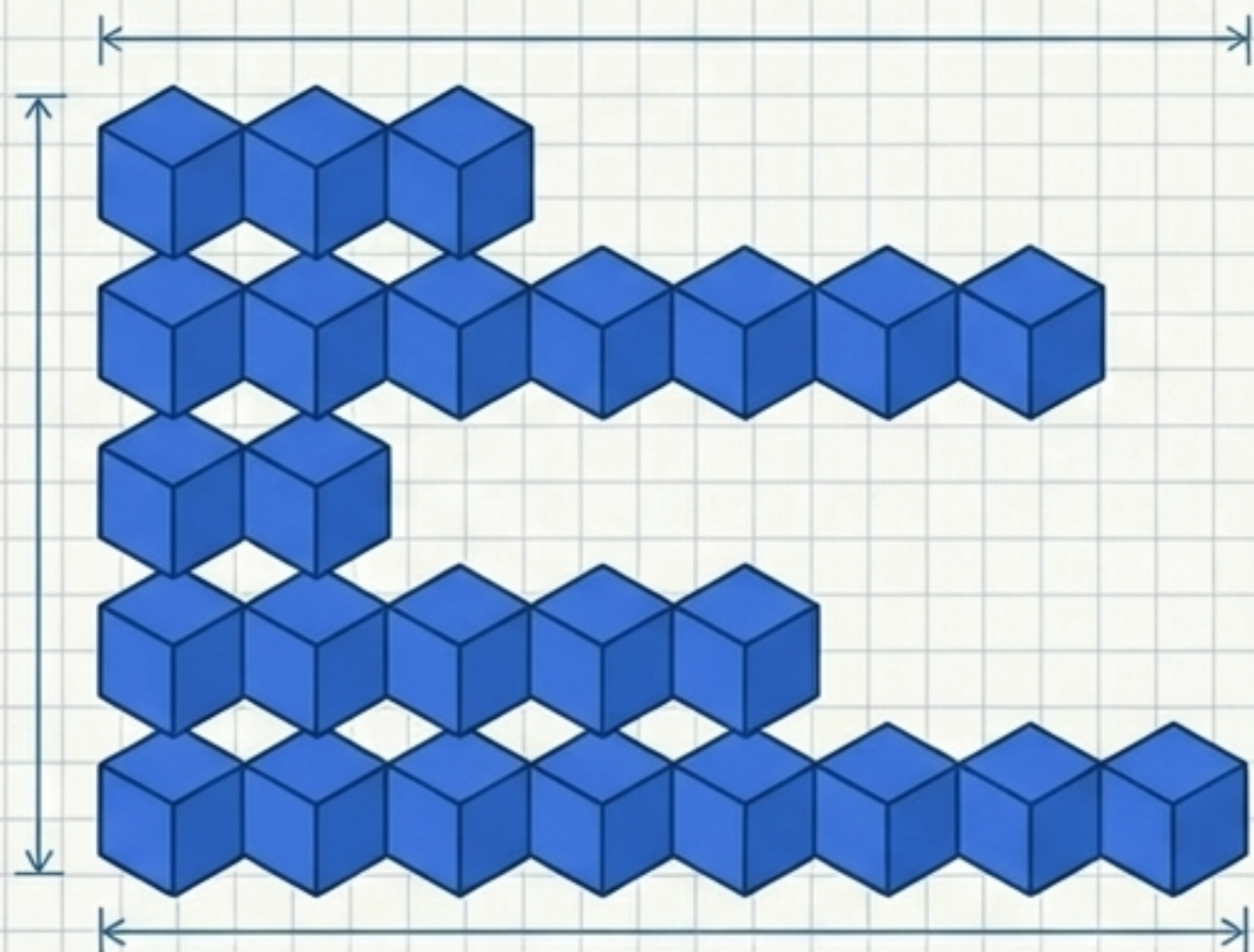
Почему " $\approx$ ",  
а не "="?

- Нормализация пробелов
- Удаление или слияние спецтокоенов

Идеальной побайтовой идентичности почти не бывает. Цель — содержательное совпадение без искажений. Это фундаментальное требование для отладки в production.

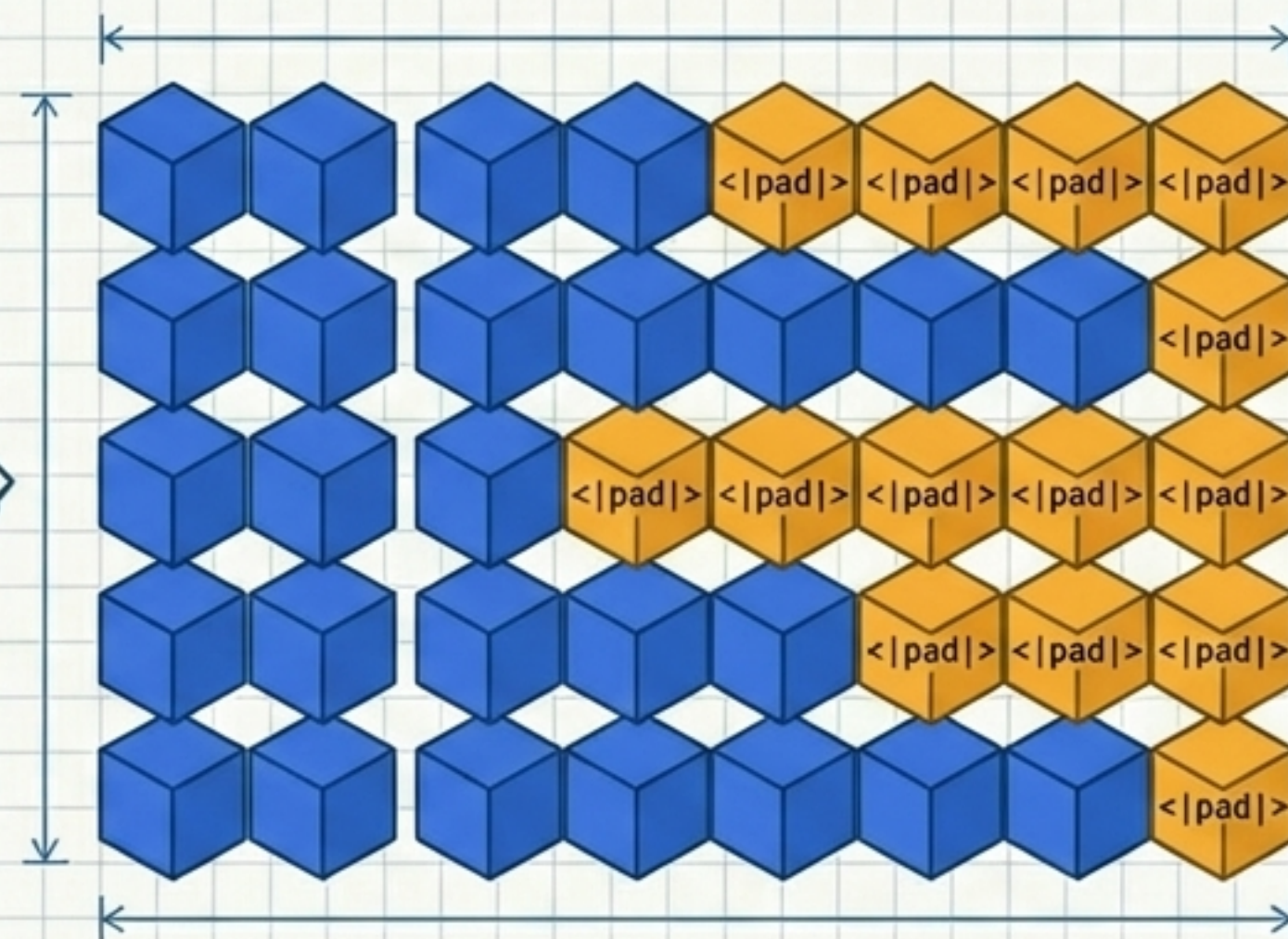
# Проблема батчей: От рваных строк к матрицам

До: Рваный массив



Выравнивание  
(Padding)

После: Тензор

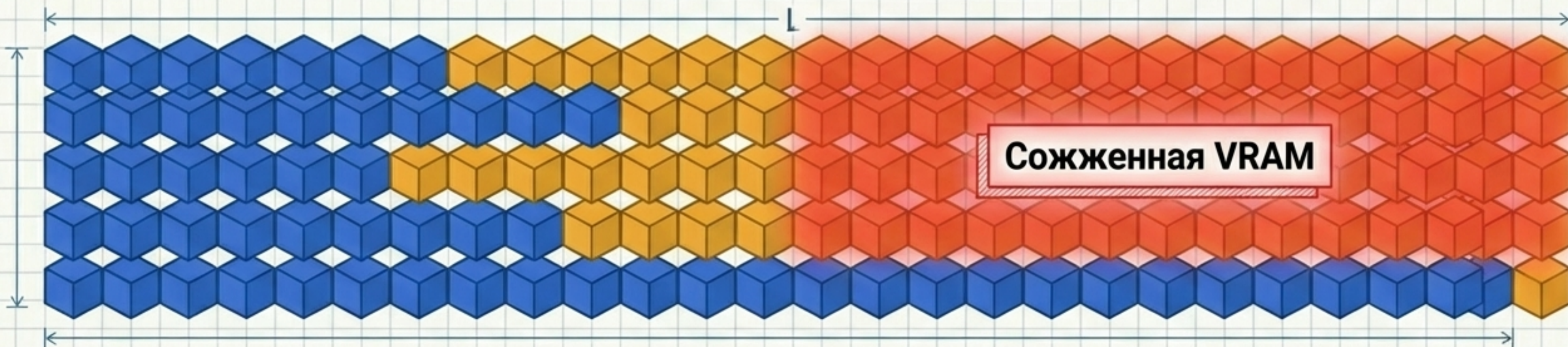


Нейросети требуют тензоров фиксированной формы.  
pad\_token\_id — обязательный элемент для объединения последовательностей разной длины в эффективный батч.

# Математика потерь: Скрытая стоимость паддинга

$$\text{Memory} \approx B \times L \times 4 \text{ bytes}$$

$B$  = Размер батча  
 $L$  = Максимальная длина в батче  
4 bytes = Вес одного токена (torch.int32)



**Одна anomalно длинная строка раздувает весь батч. Лишний паддинг — это впустую потраченная память и замедленный инференс.**

# Прагматичный Fallback: Когда нет pad\_token



С теоретической точки зрения pad и eos — разные сущности. С инженерной — контролируемый fallback спасает пайплайн от падения при сборке батча.

# Зоопарк входов против Единого Адаптера



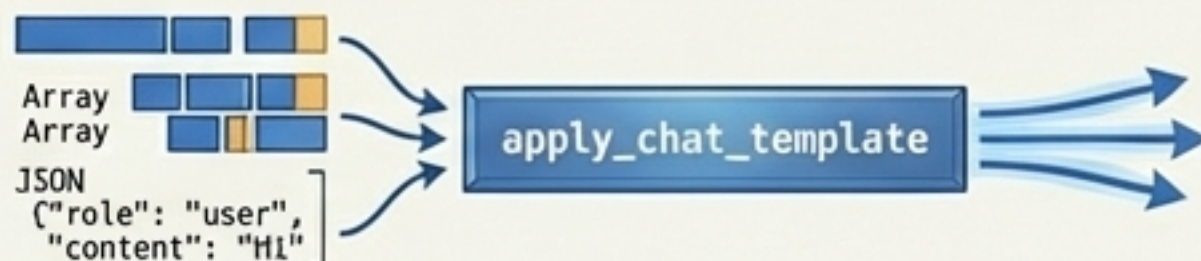
Адаптер не изобретает новую токенизацию. Он стандартизирует поведение и устраняет расхождения между реализациями перед входом в LLM.

# Инженерный контракт Адаптера



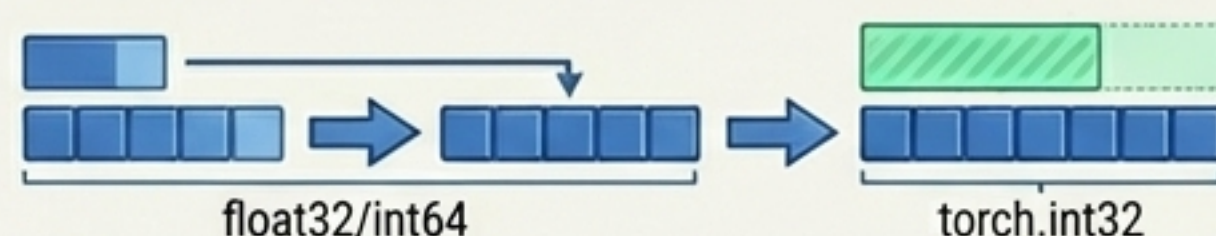
## Унификация входов

Единообразная обработка строк, батчей и диалогов через `apply_chat_template`.



## Строгая типизация

Жесткий кастинг выходов в `torch.int32` для экономии памяти и совместимости с рантаймом.



## Контроль служебных токенов

Гарантированное присутствие `'eos'` и безопасный fallback для `'pad'`.



## Экосистемность

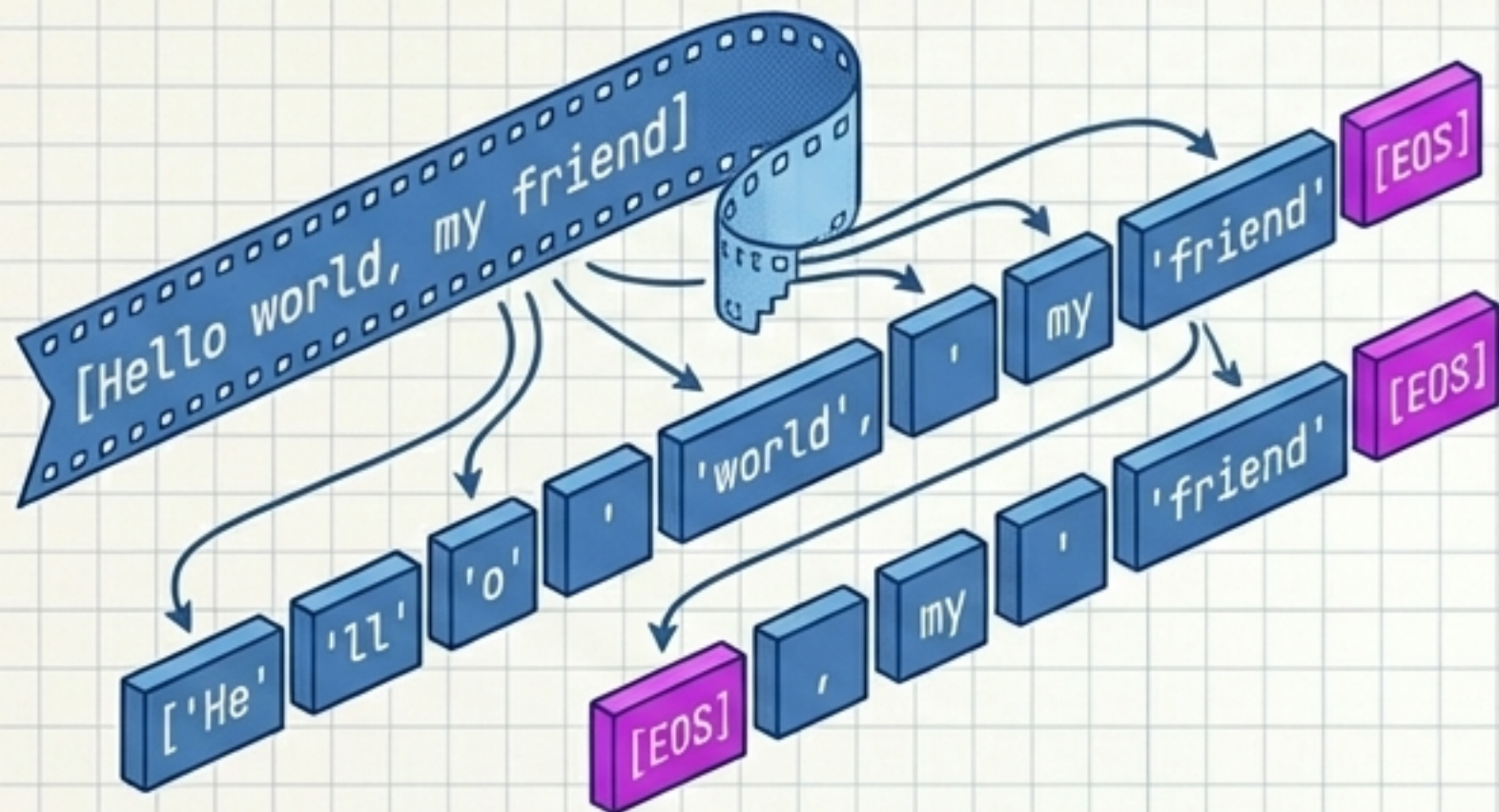
Полная совместимость с контрактами Hugging Face и архитектурами типа `'Qwen3'`.



**Вывод:** Адаптер превращает токенизацию из набора неявных соглашений в строгий **API-контракт**.

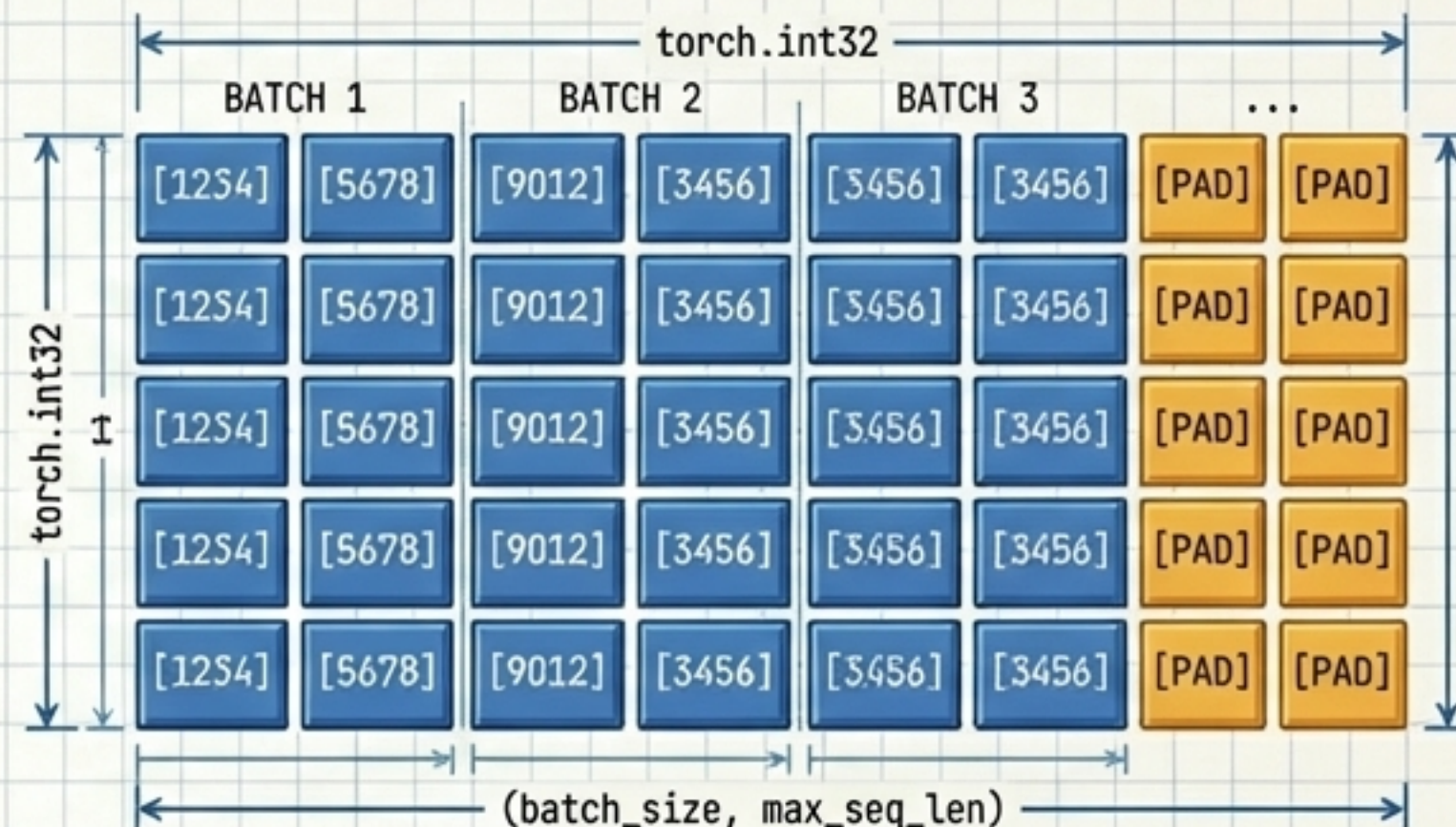
# Смысл и Стабильность

## Токенизация (Лингвистика)



- Что именно понимает модель
- Семантика словаря
- Структура диалога

## Адаптер (Инженерия)



- Как безопасно работает система
- Строгая типизация
- Контроль памяти и батчей

**Итог:** Токенизация определяет, в каком виде смысл текста попадет в нейросеть. Адаптер гарантирует, что в **production** это произойдет предсказуемо и без сбоев.